

```

# THIS IS server.py FILE
import socket #https://www.geeksforgeeks.org/socket-programming-multi-threading-python/
import _pickle as pickle
from _thread import *
import math, time

HOST = "192.168.1.23"#"10.10.108.64" #"192.168.1.23" #socket.gethostname()#HOST =
"192.168.56.1" # Standard loopback interface address (localhost)
PORT = 12345 # Port to listen on (non-privileged ports are > 1023)

print(HOST)

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

try:
    server.bind((HOST, PORT))
except socket.error as problem:
    print(str(problem))
    quit()

# GAME VARS
sWidth = 1800 #675#1800
sHeight = 800 #300#800

winner = None

pixel_list = []

time_add = 0.1 # seconds
timer = 0 + time_add
timer_stop = False

def reset_pixel_list():
    global pixel_list
    pixel_list = []
    for i in range(sHeight):
        pixel_list.append([])
        for x in range(sWidth):
            pixel_list[-1].append(None) # None-empty, or (r,g,b)

reset_pixel_list()

# THIS IS PROBABLY THE FUNCTION TO USE AS PROCEDURE AND THE PIXEL_LIST IS THE MAIN_LIST
def update_pixel_list(x,y,radius,color,circle=True): # remember since pixel_list contains
every pixel the x and y will correspond to indexvalues
    global pixel_list

    for x_count in range(radius * 2 + 1): # plus 1 because the range(goes from 0 to radius*2-1)
        for y_count in range(radius * 2 + 1):
            x_index = x - (radius - x_count)
            y_index = y - (radius - y_count) # without the if, it would just assign pixels in a
square not a circle
            if circle:
                if math.sqrt((radius-x_count)**2 + (radius-y_count)**2) <= radius \
                    and x_index >= 0 and y_index >= 0 and x_index < sWidth and y_index < sHeight: #sHeight
is len(pixel_list)-1
                    pixel_list[y_index][x_index] = color
            else:
                if x_index >= 0 and y_index >= 0 and x_index < sWidth and y_index < sHeight:
                    pixel_list[y_index][x_index] = color

def initial_pixel_setup(id, color):

```

```

radius = round(sHeight / 4)
if id % 2 == 0:
    x = sWidth
    y = round(sHeight / 2)
    update_pixel_list(x, y, radius, color, circle=False)
else:
    x = 0
    y = round(sHeight / 2)
    update_pixel_list(x, y, radius, color, circle=False)

for player in id_to_shapes:
    if player != id:
        id_to_shapes[player].append([x, y, radius, color, True])

def getTotalArea(color):
    area = 0
    for i in range(sHeight):
        for x in range(sWidth):
            if pixel_list[i][x] == color:
                area += 1
    return area

def check_for_winner(id, color):
    global winner
    if id % 2 == 0:
        for row in pixel_list:
            if row[0] == color:
                winner = id
    else:
        for row in pixel_list:
            if row[-1] == color:
                winner = id

def server_thread(*args):
    global timer
    global timer_stop

    while True:
        if winner != None:
            timer_stop = True

        if not timer_stop:
            timer = round(timer + time_add, 1)
            time.sleep(time_add)

def client_thread(user, id):
    global clients, winner

    data = user.recv(64)
    data = data.decode("utf-8").split(" ")

    name, color_string = data
    color = color_string.split(",")

    user.send(str(id).encode())

    initial_pixel_setup(id, color)

    weapons = {"paintbrush": 100, "paintballoon": 2, "paintfall": 2}
    paint_radius = round(sWidth / 180)
    balloon_size = round(sWidth / 36)
    paintfall_width = round(sWidth / 72)

```

```

paint_time = 1
balloon_time = 20
fall_time = 25

last_timer = timer

while True:
    try:
        data = user.recv(2048 * 4) # list of lists

        if not data: # if no data is being recieved, user has disconnected from server
            break

        reply = pickle.loads(data)

        if winner == None:
            update_list = reply[0]
            for item in update_list:
                x,y,radius = item[0], item[1], item[2]
                update_pixel_list(x, y, radius, color, circle=True)

                # if item[2] == paint_radius:
                #     weapons["paintbrush"] -= 1
                # elif item[2] == balloon_size:
                #     weapons["paintballoon"] -= 1
                # elif item[2] == paintfall_width:
                #     weapons["paintfall"] -= 1

            for player in id_to_shapes:
                if player != id:
                    id_to_shapes[player].append([item[0], item[1], item[2], color])

            allowed_pixel = None
            test_pixel = reply[1]
            weapon = reply[2]
            if test_pixel != None:
                if pixel_list[test_pixel[0] - 1][test_pixel[1] - 1] == color:
                    if weapons[weapon] > 0:
                        allowed_pixel = test_pixel[1], test_pixel[0]
                        weapons[weapon] -= 1

                    if weapon == "paintfall":
                        weapons["paintbrush"] += 15

            check_for_winner(id, color)

            missing_shapes = None
            id_shapes_length = len(id_to_shapes[id])
            if id_to_shapes[id] != []:
                missing_shapes = id_to_shapes[id]
                id_to_shapes[id] = id_to_shapes[id][id_shapes_length - 1:-1]
            else:
                allowed_pixel = None
                missing_shapes = None

        # timer:
        if last_timer != timer:
            if timer % paint_time == 0 and weapons["paintbrush"] < 200:
                weapons["paintbrush"] += 5
            if timer % balloon_time == 0 and weapons["paintballoon"] < 4:
                weapons["paintballoon"] += 1
            if timer % fall_time == 0 and weapons["paintfall"] < 5:
                weapons["paintfall"] += 1
            last_timer = timer

        area = None
        if winner != None and area == None:

```

```

        area = getTotalArea(color)

    win_info = (winner, area)

    relay_data = pickle.dumps(
        (allowed_pixel, missing_shapes, win_info, weapons, timer))

    user.send(relay_data)

except Exception as error:
    print("Error:", error)
    break

del clients[id]
del id_to_shapes[id]
user.close()
print(f"USER CLOSED [id #{id}]")

def reset_game():
    global timer, winner, timer_stop

    reset_pixel_list()
    timer = 0
    winner = None
    timer_stop = False

server.listen()

clients = {}  #{1:"127.0.0.1"} # id to ip

global_id = 1
id_to_user = {}  #1:"name"

id_to_shapes = {}  # 1:[["x, y, radius, color]]

first_player = None  # socket, id
while True:

    user_socket, address = server.accept()
    print(address[0] + " connected")

    clients[global_id] = address
    id_to_shapes[global_id] = []

    if global_id % 2 == 0:
        reset_game()

        start_new_thread(client_thread, (first_player[0], first_player[1]))
        start_new_thread(
            client_thread, (user_socket, global_id)
        )  # system of threading to handle individual clients inspired from
https://github.com/techwithtim/Agar-IO
        start_new_thread(server_thread, (None, None))
        first_player = None
    else:
        first_player = user_socket, global_id

    global_id += 1

```

```

# THIS IS main.py FILE (NEW FILE)

#!/usr/bin/env python
import kivy, random, math
from functools import partial

import socket
import _pickle as pickle

from kivy.config import Config

sWidth = 1800 #675#1800
sHeight = 800 #300#800

Config.set('graphics', 'width', str(sWidth))
Config.set('graphics', 'height', str(sHeight))
Config.set('graphics', 'resizable', False)

from kivy.app import App
from kivy.uix.relativelayout import RelativeLayout
from kivy.uix.gridlayout import GridLayout
from kivy.uix.floatlayout import FloatLayout
from kivy.properties import StringProperty
from kivy.uix.button import Button
from kivy.uix.widget import Widget
from kivy.core.window import Window
from kivy.properties import Clock
from kivy.graphics import Ellipse, Line, Rectangle
from kivy.uix.image import Image
from kivy.graphics.context_instructions import Color
from kivy.uix.label import Label
from kivy.uix.textinput import TextInput
from kivy.uix.screenmanager import ScreenManager, Screen

# game format inspired from GALAXY KIVY PROJECT by Jonathan Roux
# https://codewithjonathan.net/resourceskivy

#help from kivy documentation and paintapp tutorial on kivy website
(https://kivy.org/doc/stable/tutorials/firstwidget.html)
class MainWidget(FloatLayout):
    #import user_actions #https://stackoverflow.com/questions/7336802/how-to-avoid-circular-
    imports-in-python
    from user_actions import on_touch_down, on_touch_move

    def __init__(self, **kwargs):
        super(MainWidget, self).__init__(**kwargs)

        self.app = app
        self.my_id = my_id

        self.color = color

        self.weapons = {"paintbrush": 0, "paintballoon": 0, "paintfall": 0}
        self.cursor_types = ["paintbrush", "paintballoon", "paintfall"]

        self.cursor_location = Window.mouse_pos
        self.cursor_type = "paintbrush"

        self.paint_radius = round(sWidth / 180) #5

        #self.lines = [] # for server code

        # self.cursor_image = Image()
        # self.add_widget(self.cursor_image)
        #
        self.balloon_size = round(sWidth / 36)

```

```

self.balloon_init_y_speed = round(sHeight / 80)
if my_id % 2 == 0: self.balloon_init_x_speed = -round(sWidth / 180)
else: self.balloon_init_x_speed = round(sWidth / 180)
self.gravity = -1
self.balloons = []

self.balloon_offset_range = (round(sWidth / 50), round(sHeight / 53))

self.paintfall_width = round(sWidth / 72)
self.paintfall_speed = self.gravity * (sHeight / 160)
self.paintfall_swirl_factor = round(sWidth / 360)
if my_id % 2 == 0: self.paintfall_dir = -1
else: self.paintfall_dir = 1
self.paintfalls = []

self.win = [None, None]

self.data = [[], None, self.cursor_type]
self.allowed_pixel = None
self.request_pixel = None

self.missing_shapes = None

self.game_over_ran = False

self.initial_setup()   ###IMPORTANT###

self.FPS = 120
Clock.schedule_interval(self.update, 1 / self.FPS)

def initial_setup(self):
    size = round(sHeight / 2)
    if my_id % 2 == 0:
        r, g, b = self.color
        with self.canvas.before:
            Color(r, g, b)
            Rectangle(pos=(sWidth - size / 2, sHeight / 2 - size / 2),
                      size=(size, size))
    else:
        r, g, b = self.color
        with self.canvas.before:
            Color(r, g, b)
            Rectangle(pos=(0 - size / 2, sHeight / 2 - size / 2),
                      size=(size, size))

def create_balloon(self, x, y):
    size_x = self.balloon_size / sWidth
    size_y = self.balloon_size / sHeight
    self.balloons.append({
        "obj":
            Image(source="Tool Images/paintballoon.png",
                  pos=(x - (size_x * sWidth) / 2, y - (size_y * sHeight) / 2),
                  size_hint=(size_x, size_y)),
        "sy":
            self.balloon_init_y_speed,
        "sx":
            self.balloon_init_x_speed
    })

def create_balloon_splat(self, x, y):
    random_x_offset = random.randint(-self.balloon_offset_range[0],
                                      self.balloon_offset_range[0])
    random_y_offset = random.randint(-self.balloon_offset_range[1],
                                      self.balloon_offset_range[1])

    with self.canvas:
        r, g, b = self.color   https://note.nkmk.me/en/python-tuple-list-unpack/

```

```

Color(r, g, b)
x_pos = round(x + random_x_offset)
y_pos = round(y + random_y_offset)
Ellipse(pos=(x_pos, y_pos),
        size=(self.balloon_size * 2, self.balloon_size *
              2)) #ellipse x,y is bottom left and size is diameter

self.data[0].append([
    round(x_pos + self.balloon_size),
    round(y_pos + self.balloon_size), self.balloon_size
])

def update_balloons(self):
    for i, balloon in enumerate(self.balloons):
        if balloon["sy"] == -self.balloon_init_y_speed: # meaning balloon has made a full jump
            and is ready to splat
            self.create_balloon_splat(balloon["obj"].x, balloon["obj"].y)
            self.balloons[i]["obj"].color = (0, 0, 0, 0)
            self.balloons[i]["obj"].remove_from_cache()
            self.balloons.remove(balloon)

        else:
            self.balloons[i]["obj"].x += balloon["sx"]
            self.balloons[i]["obj"].y += balloon["sy"]
            self.balloons[i]["sy"] += self.gravity

def create_paintfall(self, x, y):
    r, g, b = self.color
    with self.canvas:
        Color(r, g, b)
        self.paintfalls.append([
            Line(points=[x, y], width=self.paintfall_width), 1, self.paintfall_dir
        ])

    self.data[0].append([x, y, self.paintfall_width])

def update_paintfall(self):
    for i, fall in enumerate(self.paintfalls):
        last_x, last_y, swirl, dir = fall[0].points[-2], fall[0].points[
            -1], fall[1], fall[2]
        if last_y <= 0:
            self.paintfalls.remove(fall)
        else:
            # X Code
            x = round(last_x + swirl) # swirl
            if abs(swirl) >= self.paintfall_swirl_factor:
                self.paintfalls[i][2] *= -1
            self.paintfalls[i][1] += self.paintfalls[i][2]

            #Y Code
            y = round(last_y + self.paintfall_speed)
            self.paintfalls[i][0].points = self.paintfalls[i][0].points + [x, y]
            self.data[0].append([x, y, self.paintfall_width, self.color])

def drawShapes(self):
    x, y = self.allowed_pixel
    with self.canvas:
        if self.cursor_type == "paintbrush":
            #if self.pixel_list[round(touch.y) - 1][round(touch.x) - 1] == self.color:
            r, g, b = self.color
            Color(r, g, b)
            # don't divide the self.paint_radius by 2 to center because we need the radius by 2 to
            get a big enough circle
            x_pos, y_pos = round(x - self.paint_radius), round(y -
                                                                    self.paint_radius)
            Ellipse(pos=(x_pos, y_pos),
                    size=(self.paint_radius * 2, self.paint_radius * 2))

```

```

        self.data[0].append([
            round(x_pos + self.paint_radius),
            round(y_pos + self.paint_radius), self.paint_radius, self.color
        ])

    elif self.cursor_type == "paintballoon":
        self.create_balloon(x, y)

    elif self.cursor_type == "paintfall":
        self.create_paintfall(x, y)

def addMissingShapes(self):
    for shape in self.missing_shapes:
        x, y, radius, color = shape[0], shape[1], shape[2], shape[3]
        if len(shape) < 5:
            with self.canvas:
                r, g, b = color
                Color(r, g, b)
                x_pos, y_pos = round(x - radius), round(y - radius)
                Ellipse(pos=(x_pos, y_pos), size=(radius * 2, radius * 2))
        else:
            with self.canvas:
                r, g, b = color
                Color(r, g, b)
                x_pos, y_pos = round(x - radius), round(y - radius)
                Rectangle(pos=(x_pos, y_pos), size=(radius * 2, radius * 2))

def game_over(self):
    percentage = self.win[1] / (sWidth * sHeight) * 100 #self.win[1] is area
    percentage = round(percentage, 2)
    if self.win[0] == my_id:
        display_text = "You Won!"
    else:
        display_text = "You lost..."

    app.screen_manager.current = "GameOver"

    app.game_over.label1.text = display_text
    app.game_over.label2.text = f"Covered {percentage}% of screen!"

def clear_data(self):
    self.data = [[], None, self.cursor_type]

def update(self, dt):
    time_factor = dt * self.FPS

    if self.win[0] == None:
        self.update_balloons()
        self.update_paintfall()

    self.data[1] = self.request_pixel
    self.data[2] = self.cursor_type

    data = pickle.dumps((self.data[0], self.data[1], self.data[2]))
    client_socket.send(data)

    self.clear_data()

    recv_data = client_socket.recv(2048 * 4)
    reply = pickle.loads(recv_data)
    self.allowed_pixel, self.missing_shapes, self.win, self.weapons, self.timer = reply[
        0], reply[1], reply[2], reply[3], reply[4]

    for i, weapon in enumerate(self.weapons):
        app.button_label_list[i].text = str(self.weapons[weapon])

    app.update_timer(self.timer)

```

```

        if self.allowed_pixel != None:
            self.drawShapes()

        if self.missing_shapes != None:
            self.addMissingShapes()

        self.allowed_pixel = None
        self.request_pixel = None

    else:
        if not self.game_over_ran:
            self.game_over()
            self.game_over_ran = True

    #f"{area} pixels: {area/(sWidth*sHeight) * 100}% of screen"

```

```

class MainMenu(GridLayout):

```

```

    def __init__(self, **kwargs):
        super().__init__(**kwargs)

        self.cols = 1

        self.fields_layout = GridLayout(cols=2)

        self.ip_label = Label(text="SERVER IP: ")
        self.ip_text_input = TextInput(hint_text="127.0.0.1",
                                       multiline=False,
                                       text="127.0.0.1")

        self.color_label = Label(text="RGB Color: ")
        self.color_input = TextInput(hint_text="0,0,255",
                                     multiline=False,
                                     text="1,0,0")

        self.fields_layout.add_widget(self.ip_label)
        self.fields_layout.add_widget(self.ip_text_input)
        self.fields_layout.add_widget(self.color_label)
        self.fields_layout.add_widget(self.color_input)

        self.join_button = Button(text="Join Game!")
        self.join_button.bind(on_release=self.temporary_func)

        self.add_widget(self.fields_layout)
        self.add_widget(self.join_button)

    def temporary_func(self, obj):
        app.screen_manager.current = "Loading"
        Clock.schedule_once(self.connect_to_server, 1)

    def connect_to_server(self, obj):
        global my_id, client_socket, PORT, color
        self.server_ip = self.ip_text_input.text
        self.color_string = self.color_input.text

        # NETWORK/SOCKET PART
        PORT = 12345

        client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        client_socket.connect((self.server_ip, PORT))

        name = "Lol"

        color_list = self.color_string.split(",")
        color = int(color_list[0]), int(color_list[1]), int(color_list[2])

```

```

data = name + " " + self.color_string
client_socket.send(data.encode())

recv_data = client_socket.recv(1024)
my_id = int(recv_data.decode())
print(my_id)

app.create_main_page()
app.screen_manager.current = "MainWidget"

```

```

class LoadingPage(GridLayout):

```

```

    def __init__(self, **kwargs):
        super().__init__(**kwargs)

        self.cols = 1

        self.dots = 0
        self.label = Label(text="Waiting for a 2nd player to join")

        self.add_widget(self.label)

        self.FPS = 3
        Clock.schedule_interval(self.update, 1 / self.FPS)

    def update(self, dt):
        if self.dots < 3:
            self.label.text = self.label.text + "."
            self.dots += 1
        else:
            self.label.text = "Waiting for a 2nd player to join"
            self.dots = 0

```

```

class GameOverPage(GridLayout):

```

```

    def __init__(self, **kwargs):
        super().__init__(**kwargs)

        self.cols = 1

        self.label1 = Label(text="")
        self.label2 = Label(text="")

        self.button = Button(text="Menu")
        self.button.bind(on_release=self.go_to_menu)

        self.add_widget(self.label1)
        self.add_widget(self.label2)
        self.add_widget(self.button)

    def go_to_menu(self, obj):
        app.clear_canvas()
        app.screen_manager.current = "Menu"

```

```

class PaintBattleApp(App):

```

```

    def build(self):
        self.screen_manager = ScreenManager()

        self.menu = MainMenu()
        self.menu_screen = Screen(name="Menu")
        self.menu_screen.add_widget(self.menu)
        self.screen_manager.add_widget(self.menu_screen)

```

```

self.loading = LoadingPage()
self.loading_screen = Screen(name="Loading")
self.loading_screen.add_widget(self.loading)
self.screen_manager.add_widget(self.loading_screen)

self.game_over = GameOverPage()
self.game_over_screen = Screen(name="GameOver")
self.game_over_screen.add_widget(self.game_over)
self.screen_manager.add_widget(self.game_over_screen)

return self.screen_manager

def create_main_page(self):
    self.main_widget = MainWidget()

    self.button_layout = FloatLayout(size=(
        sWidth,
        sHeight)) # https://kivy.org/doc/stable/api-kivy.uix.floatlayout.html

    self.button_size = sWidth / 18
    self.create_tool_buttons(
        len(self.main_widget.cursor_types),
        self.button_size) # create a button for each tool in cursor_types list

    size_x, size_y = round(sWidth / 5), round(sHeight / 8)
    self.timer_label = Label(size_hint=(size_x / sWidth, size_y / sHeight),
                             pos=(sWidth / 2, sHeight - size_y),
                             text="0:00",
                             text_size=(size_x, size_y),
                             color=(1, 1, 1))
    self.button_layout.add_widget(self.timer_label)

    self.main_screen = Screen(name="MainWidget")
    self.main_screen.add_widget(self.main_widget)
    self.main_screen.add_widget(self.button_layout)
    self.screen_manager.add_widget(self.main_screen)

def create_tool_buttons(self, button_num, size):
    self.button_list = []
    self.button_label_list = []
    if my_id % 2 == 0:
        start_x = sWidth - size
        dir = -1
    else:
        start_x = 0
        dir = 1

    for i in range(button_num):
        self.button_list.append(
            Button(
                size_hint=(size / sWidth, size / sHeight),
                pos=(start_x + (dir * i * size), 0),
                background_normal=
                    f"Tool Images/{self.main_widget.cursor_types[i]}.png",
#https://www.geeksforgeeks.org/use-image-as-a-button-in-kivy/
                border=(
                    0, 0,
                    0, 0
                ), # to fix distortion: https://stackoverflow.com/questions/34727938/image-
                #distorted-when-widget-size-differs-from-dimensions-of-stored-image
                #background_down = 'down.png', #do later
            ))

        callback = partial(
            self.button_functionality, self.main_widget.cursor_types[i]
        ) # got help from stack overflow with this

```

```
link:https://stackoverflow.com/questions/33586688/kivy-button-binding-function-with-argument
self.button_list[-1].bind(on_release=callback)

self.button_label_list.append(
    Label(size_hint=(size / sWidth, (size / sHeight)),
          pos=(start_x + (dir * i * size), size * (2 / 3)),
          text=str(
              self.main_widget.weapons[self.main_widget.cursor_types[i]])))

self.button_layout.add_widget(self.button_list[-1])
self.button_layout.add_widget(self.button_label_list[-1])

def button_functionality(self, *args): # **args maybe
    self.main_widget.cursor_type = args[0]
    print(self.main_widget.cursor_type)

def update_timer(self, time):
    minutes = int(time / 60)
    seconds = int(time - minutes * 60)
    if seconds < 10:
        seconds = f"0{seconds}"

    text = f"{minutes}:{seconds}"

    self.timer_label.text = text

def clear_canvas(self):
    self.main_widget.canvas.clear()

my_id = None
client_socket = None

app = PaintBattleApp()
app.run()
#client_socket.close()
```

```
# THIS IS user_actions.py FILE (NEW FILE)
```

```
import math
from kivy.graphics import Ellipse, Line
from kivy.graphics.context_instructions import Color
from kivy.uix.widget import Widget
#import main
```

```
sWidth = 1800 #675#1800
sHeight = 800 #300#800
```

```
def on_touch_down(self, touch):
    # if self.my_id % 2 == 0:
    #     if touch.x < sWidth and touch.x > (sWidth - self.app.button_size *
len(self.cursor_types)):
    #         if touch.y < self.app.button_size and touch.y > 0:
    #             print("yo")
    #             return super(Widget, self).on_touch_down(touch)
    # else:
    #     if touch.x < (self.app.button_size * len(self.cursor_types)) and touch.x > 0:
    #         if touch.y < self.app.button_size and touch.y > 0:
    #             print("yo")
    #             return super(RelativeLayout, self).on_touch_down(touch)

    if self.win[0] == None:
        if self.cursor_type == "paintbrush":
            touch.ud['start_pos'] = (
                touch.x, touch.y
            ) # https://stackoverflow.com/questions/51841557/kivy-what-does-touch-ud-means
            #touch.ud['line'] = Line(points=(touch.x, touch.y), width=self.paint_radius) # from
https://kivy.org/doc/stable/tutorials/firstwidget.html
            #self.lines.append([(touch.x, touch.y)]) # for server grade
            self.request_pixel = (round(touch.y), round(touch.x))
```

```
def on_touch_move(self, touch):
    if self.win[0] == None:
        if self.cursor_type == "paintbrush":
            self.request_pixel = (round(touch.y), round(touch.x))
            if (round(touch.y), round(touch.x)) == self.allowed_pixel:
                if 'start_pos' in touch.ud:
                    x_dist, y_dist = touch.x - touch.ud['start_pos'][
                        0], touch.y - touch.ud['start_pos'][1]
                    dist = math.sqrt(x_dist**2 + y_dist**2)
                    angle = math.atan2(
                        y_dist, x_dist
                    ) # https://stackoverflow.com/questions/283406/what-is-the-difference-between-atan-
and-atan2-in-c
                    x_step = math.cos(
                        angle
                    ) * self.paint_radius # if dist>self.paint_radius else math.cos(angle)
                    y_step = math.sin(
                        angle
                    ) * self.paint_radius # if dist>self.paint_radius else math.sin(angle) # the
*radius is for optimization
                    for i in range(
                        round(dist /
                            self.paint_radius if dist > self.paint_radius else 1)):
                        x_pos = round(touch.ud['start_pos'][0] + i * x_step -
                            self.paint_radius) if x_dist != 0 else round(
                            touch.ud['start_pos'][0]) - self.paint_radius
                        y_pos = round(touch.ud['start_pos'][1] + i * y_step -
                            self.paint_radius) if y_dist != 0 else round(
                            touch.ud['start_pos'][1]) - self.paint_radius

                        with self.canvas:
```

```
    r, g, b = self.color
    Color(r, g, b)
    Ellipse(pos=(x_pos, y_pos),
             size=(self.paint_radius * 2, self.paint_radius * 2))

    self.data[0].append([
        round(x_pos + self.paint_radius),
        round(y_pos + self.paint_radius), self.paint_radius
    ])

    touch.ud['start_pos'] = (
        touch.x, touch.y
    ) # new start point is at the end of the final ellipse

    #touch.ud['line'].points += [touch.x, touch.y]
    #self.lines[-1].append((touch.x, touch.y)) # for server code
    #print(self.lines)
```